

Full-Stack Email Job Scheduler

ReachInbox.ai (Outbox Labs) — Software Development Intern Assignment
Candidate Reference Guide

Deadline: **48 hours**
Backend: **TypeScript + Express + BullMQ + Redis + MySQL/Postgres**
Frontend: **React/Next.js + Tailwind + TypeScript**

1. Objective

Build a production-grade email scheduler service and dashboard — a small slice of what ReachInbox runs in production. The system accepts email-send requests through an API, schedules them for a specific time using BullMQ and Redis (never cron), sends them through a fake SMTP provider (Ethereal Email), and must keep working correctly across server restarts. A frontend dashboard lets a user schedule new emails and see what's scheduled and what's already sent.

What you'll practice:

- Durable job scheduling with BullMQ + Redis, instead of cron
- Designing a relational schema that survives process restarts without duplicate or lost sends
- Rate limiting and concurrency control that stays correct across multiple workers
- Real Google OAuth login and a dashboard UI built to match a given design
- Full-stack TypeScript, from typed API responses to typed React props

2. Technical Requirements

Backend

- **TypeScript** — type safety across the API, queue, and DB layers.
- **Express.js** — the API layer that accepts scheduling requests and serves dashboard data.
- **BullMQ (Redis-backed)** — a persistent job queue for delayed jobs; this is what replaces cron entirely.
- **MySQL or PostgreSQL** — the durable source of truth for every scheduled and sent email, independent of what's currently in the queue.
- **Ethereal Email** — fake SMTP, so you can exercise a real send flow without emailing real inboxes.

Frontend

- **React.js or Next.js** — the dashboard framework.
- **Tailwind CSS (or similar)** — styling, matching the provided Figma as closely as possible.
- **TypeScript (strongly preferred)** — typed API responses and component props.

Infra

- **Docker (recommended)** — run Redis and your DB consistently without installing them locally.

3. Project Features

Backend — core scheduler behavior

- Accept scheduling requests via API and store them in the relational DB
- Schedule sends using BullMQ delayed jobs (or a custom Redis/DB-based scheduler) — never cron
- Send from multiple senders via Ethereal Email
- Survive restarts: future emails still send on time, and nothing is duplicated or restarted from scratch

Backend — throughput, rate limiting & concurrency (required)

- **Worker concurrency** — configurable, and safe when many jobs run in parallel.
- **Delay between sends** — a minimum gap between individual emails (BullMQ limiter or custom delay), documented in the README.

- **Hourly rate limit** — configurable via env (global or per-sender), enforced with Redis/DB-backed counters — never in-memory only, since that breaks across multiple workers.
- **On limit hit** — reschedule the job into the next hour window; never drop or permanently fail it.
- **Under load** — your design must define what happens with 1000+ emails scheduled for the same time, even though you don't need to actually send that many through Ethereum.

Hard Constraints — non-negotiable

No cron, in any form — no OS crontab, no node-cron, no agenda. Scheduling only via BullMQ delayed jobs or a custom Redis/DB scheduler. The system must persist across restarts, and the same email must never be sent twice — idempotency is required, not optional.

Frontend — features

- **Google login (required)** — real OAuth, not a mock; redirect to the dashboard after login; show name, email, and avatar in the header; provide logout.
- **Main dashboard** — header + Scheduled/Sent tabs + a primary “Compose New Email” button, following the Figma layout.
- **Compose flow** — subject, body, CSV/text upload of leads (parsed, with the detected address count shown), start time, delay between emails, hourly limit, and a Schedule action that calls the backend API.
- **Scheduled Emails table** — email, subject, scheduled time, status, with loading and empty states.
- **Sent Emails table** — email, subject, sent time, status (sent/failed), with loading and empty states.
- **Code quality** — clean folder structure, reusable components (buttons, inputs, tables, modals), DRY code, typed API responses and props, and basic error handling (messages/toasts).

4. Step-by-Step Implementation Guide

1. **Set up infra.** Start Redis and your DB, ideally via Docker, and confirm BullMQ can connect. Everything else depends on a running queue and a durable database.
2. **Design the DB schema.** At minimum, a table for scheduled emails: recipient, subject, body, sender, send time, and status. The DB is your source of truth if the process restarts — don't rely on the queue alone to remember state.
3. **Build the schedule API.** An Express endpoint that validates a request, writes a row to the DB, then enqueues a BullMQ delayed job for the right time. This is the entry point for both Postman testing and the frontend Compose flow.
4. **Build the worker, with an idempotency guard.** A BullMQ worker that sends via Ethereum and marks the DB row as sent only once, checking status before it sends. This is what satisfies the “no duplicate sends” hard constraint.
5. **Add concurrency, delay, and hourly rate limiting.** Configure worker concurrency, add a minimum delay between sends, and enforce a configurable per-hour cap with Redis/DB counters, rescheduling into the next hour instead of dropping jobs. This is the part worth the most engineering care.
6. **Test the restart scenario.** Schedule a few emails a few minutes out, stop the server, restart it, and confirm they still send on time without double-sending. You'll need to demo exactly this in your submission video.
7. **Build the frontend shell.** Google OAuth login, header with user info, and the dashboard tabs — this is the first thing evaluators will click through.
8. **Build Compose, Scheduled, and Sent views.** Wire each to your backend API, match the Figma layout, and add loading and empty states throughout.
9. **Write the README and record the demo.** Document setup, your architecture decisions, and trade-offs, then record a short video showing scheduling, the dashboard, and the restart scenario.

5. Project Structure

```
reachinbox-scheduler/
├── backend/
│   ├── src/
│   │   ├── api/
│   │   │   ├── schedule.routes.ts # POST /schedule, GET /emails...
│   │   │   └── queue/
│   │   │       ├── queue.ts # BullMQ queue setup
│   │   │       ├── worker.ts # BullMQ worker — sends via Ethereum
│   │   └── rateLimiter/
│   │       ├── hourlyLimiter.ts # Redis-backed per-hour counters
│   └── db/
```

```

├── schema.sql          # or ORM models
├── client.ts
├── index.ts            # Express app entry
├── .env.example
├── package.json
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── ComposeModal.tsx
│   │   │   ├── EmailTable.tsx
│   │   │   └── Header.tsx
│   │   ├── lib/
│   │   │   ├── api.ts          # typed API client
│   │   │   └── types/
│   │   │       └── email.ts
│   └── package.json
├── docker-compose.yml  # Redis + DB
└── README.md

```

- **backend/src/queue/** — everything that talks to BullMQ lives here, separate from the API routes.
- **backend/src/rateLimiter/** — kept isolated so the hourly-limit logic is easy to find and test on its own.
- **frontend/src/components/** — reusable pieces shared across the Compose, Scheduled, and Sent views.

6. Example Code Snippets

Enqueuing a delayed job (no cron)

```

await emailQueue.add(
  "send-email",
  { emailId },
  { delay: sendAt.getTime() - Date.now(), jobId: emailId }
);

```

delay is computed from the target send time. Using the DB row's id as jobId gives you a natural idempotency key — BullMQ won't enqueue the same job id twice.

A Redis-backed hourly counter

```

const key = `rate:${sender}:${hourWindow}`;
const count = await redis.incr(key);
if (count === 1) await redis.expire(key, 3600);
if (count > MAX_EMAILS_PER_HOUR) {
  await job.moveToDelayed(Date.now() + msUntilNextHour);
  return;
}

```

Incrementing a Redis key scoped to sender + hour keeps the count correct across multiple worker instances. Going over the limit reschedules the job instead of failing it.

Sending idempotently

```

const email = await db.email.findUnique({ where: { id: emailId } });
if (email.status === "sent") return; // already sent — skip
await sendMail(email);
await db.email.update({ where: { id: emailId }, data: { status: "sent" } });

```

Checking status before sending is what protects you if a job re-runs after a crash mid-send.

7. API Endpoints Overview

Exact routes are your choice — document whatever you build in the README. A reasonable shape:

- **POST /api/emails/schedule** — create and enqueue a batch of scheduled emails
- **GET /api/emails/scheduled** — list emails not yet sent
- **GET /api/emails/sent** — list emails with status sent or failed
- **GET /api/auth/google** and **/api/auth/callback** — the OAuth login flow

8. Error Handling & Edge Cases

- **1000+ emails scheduled at once** — enqueue without blocking the API response; let the worker spread out the actual sending.
- **Hourly limit reached** — reschedule into the next window; never drop or fail the job.
- **Server restart mid-queue** — rely on BullMQ's own Redis persistence first; don't re-seed jobs from the DB in a way that creates duplicates.
- **Bad rows in a CSV upload** — parse defensively, and show the user how many valid addresses were actually found.

9. What Evaluators Look At

- Correct queue wiring — BullMQ + Redis, with genuinely no cron anywhere
- Rate limiting and concurrency that stay correct across multiple workers, not just in memory
- Clean restart behavior — this gets demoed on video, so it needs to actually work
- Frontend fidelity to the Figma, plus code quality: structure, reusable components, typed API, UX states

10. Submission Guidelines

1. **Private GitHub repository** (monorepo or separate backend/frontend folders are both fine) — grant access to the reviewers named in your assignment email.
2. **README.md** covering: how to run the backend (Express, Redis, DB, BullMQ worker) and frontend, how to set up Ethereum Email and env variables, an architecture overview (scheduling, restart persistence, rate limiting/concurrency), and a feature checklist mapped to backend and frontend requirements.
3. **A short demo video** (max 5 minutes) showing: creating scheduled emails, the dashboard with Scheduled and Sent tabs, and a restart scenario — stop the server, start it again, and show future emails still sending. Bonus: briefly show rate limiting/delay behavior under load.
4. **A note on assumptions, shortcuts, or trade-offs** you made anywhere in the build.

Deadline: 48 hours.

Note

All submitted code is reviewed for originality — plagiarism results in the assignment being rejected. Receiving this assignment already puts you ahead of many candidates. Good luck!